

23AID303 - Quantum Computing and Algorithms

Lab 4

1. Implementation of Deutsch Jozsa Algorithm

```
%matplotlib inline
import random
from qiskit import QuantumCircuit, transpile
from qiskit_aer import Aer
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

# Number of input qubits
n = 3

# -----
# ORACLE DEFINITIONS
# -----

# Constant oracle (always returns 0)
def constant_oracle(n):
    qc = QuantumCircuit(n+1)
    # Does nothing (always 0)
    return qc

# Constant oracle (always returns 1)
def constant_oracle_one(n):
    qc = QuantumCircuit(n+1)
    qc.x(n) # flip output qubit, so f(x)=1
    qc.barrier()
    return qc

# Balanced oracle: f(x) = XOR of all input bits
def balanced_oracle(n):
    qc = QuantumCircuit(n+1)
    for qubit in range(n):
        qc.cx(qubit, n)
    qc.barrier()
```

```

    return qc

# Random choice
oracle_choice = random.choice(["constant0", "constant1", "balanced"])
if oracle_choice == "constant0":
    oracle = constant_oracle(n)
elif oracle_choice == "constant1":
    oracle = constant_oracle_one(n)
else:
    oracle = balanced_oracle(n)

print("Oracle chosen randomly:", oracle_choice)

# -----
# DEUTSCH-JOZSA MAIN CIRCUIT
# -----

qc = QuantumCircuit(n+1, n)

# Step 1: Initialize output qubit in |1>
qc.x(n)

# Step 2: Apply Hadamards
qc.h(range(n+1))
qc.barrier()

# Step 3: Apply oracle
qc.compose(oracle, inplace=True)

# Step 4: Hadamards on input qubits
qc.h(range(n))

# Step 5: Measure input qubits
qc.measure(range(n), range(n))

qc.draw('mpl')
plt.show()

# Run on simulator
simulator = Aer.get_backend('qasm_simulator')

```

```

compiled_circuit = transpile(qc, simulator)
result = simulator.run(compiled_circuit, shots=1024).result()
counts = result.get_counts()

```

```

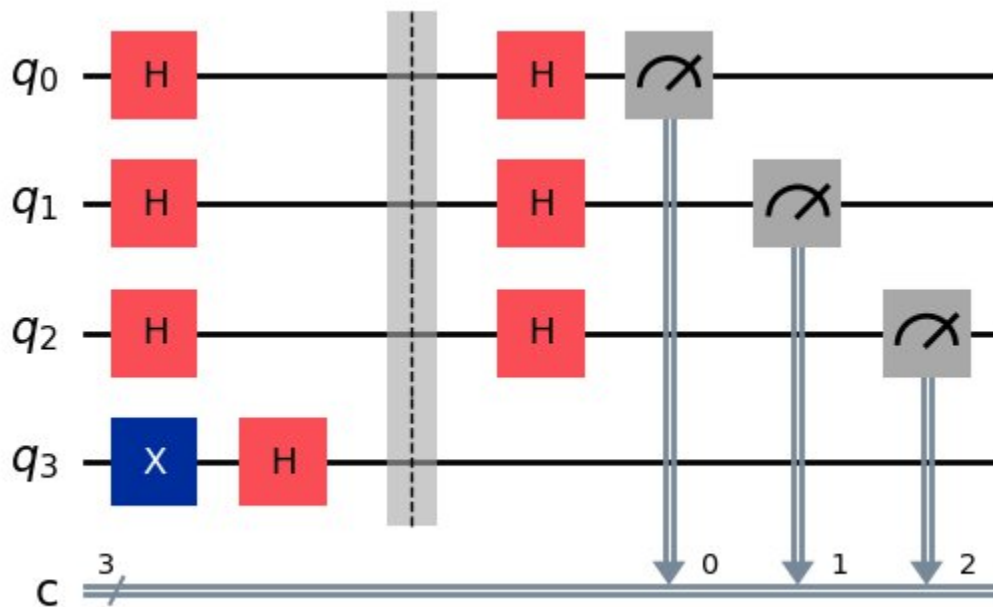
plot_histogram(counts)
plt.show()

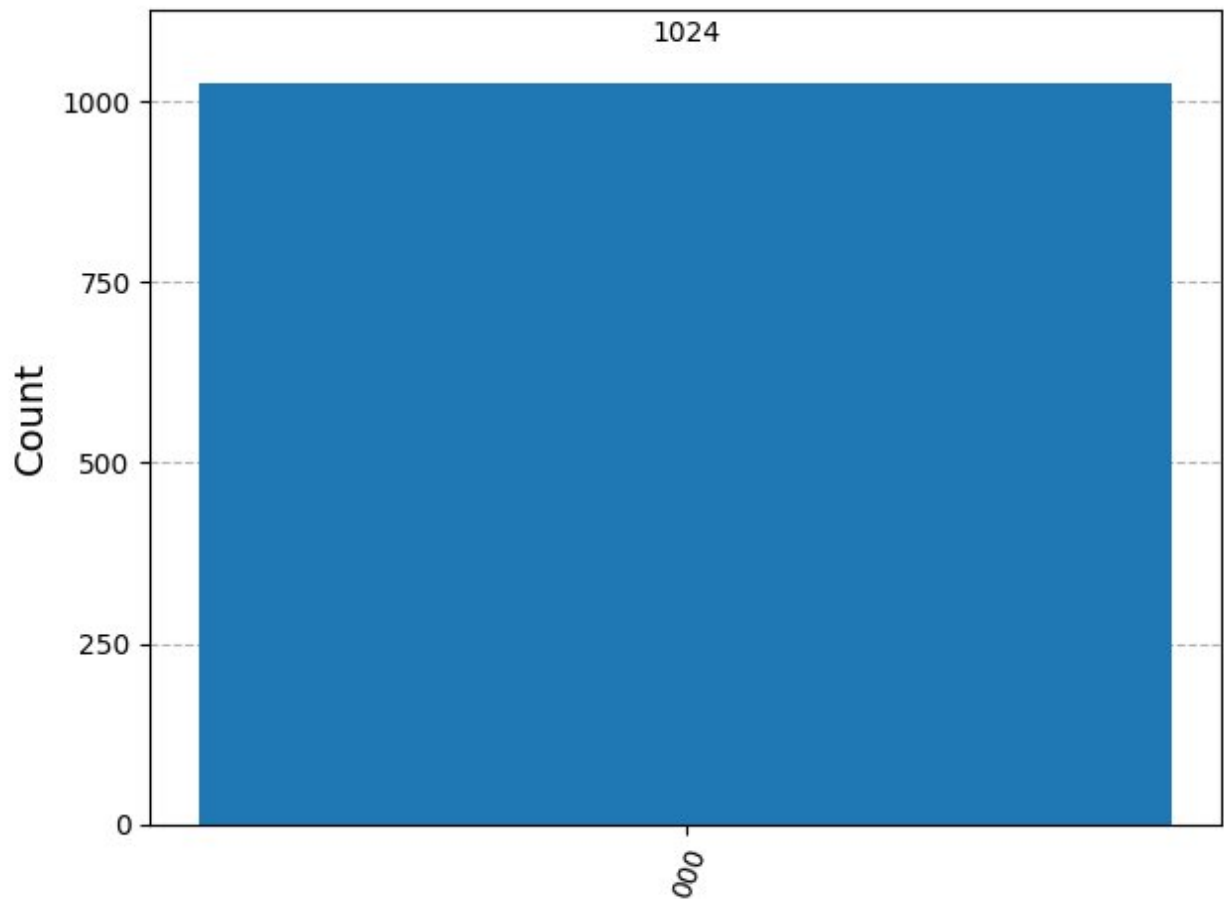
```

```

# -----
# Decision
# -----
if "0"*n in counts and counts["0"*n] == 1024:
    print("Function is CONSTANT")
else:
    print("Function is BALANCED")

```





Function is CONSTANT

2. Implementation of Bernstein-Vazirani Algorithm

```
%matplotlib inline
```

```
import random
```

```
from qiskit import QuantumCircuit, transpile
```

```
from qiskit_aer import Aer
```

```
from qiskit.visualization import plot_histogram
```

```
import matplotlib.pyplot as plt
```

```
# Bernstein-Vazirani algorithm for n=3
```

```
n = 4 # number of input qubits
```

```

# Secret string 'a' (randomly chosen here)
a = "".join([str(random.randint(0,1)) for _ in range(n)])
print("Secret string a:", a)

# -----
# ORACLE for  $f(x) = a \cdot x \bmod 2$ 
# -----
def bv_oracle(a):
    n = len(a)
    qc = QuantumCircuit(n+1)
    for i, bit in enumerate(reversed(a)): # reversed for Qiskit qubit ordering
        if bit == '1':
            qc.cx(i, n) # flip output qubit if a_i=1
    return qc

oracle = bv_oracle(a)

# Create a circuit with n input qubits + 1 output qubit
qc = QuantumCircuit(n+1, n)

# Step 1: Initialize last qubit in  $|1\rangle$ 
qc.x(n)

# Step 2: Apply Hadamards to all qubits

```

```

qc.h(range(n+1))
qc.barrier()

# ---- ORACLE DEFINITION ----
# Step 3: append the oracle
qc.compose(oracle, inplace=True)
qc.barrier()

# Step 4: Apply Hadamards to input qubits
qc.h(range(n))

# Step 5: Measure input qubits
qc.measure(range(n), range(n))

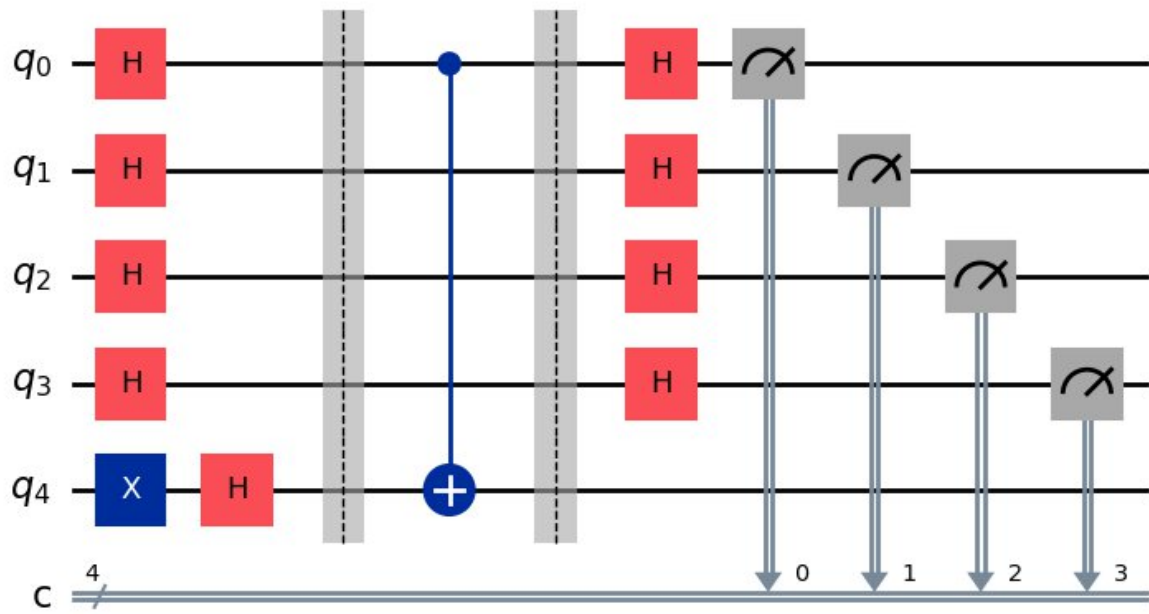
qc.draw('mpl')
plt.show()

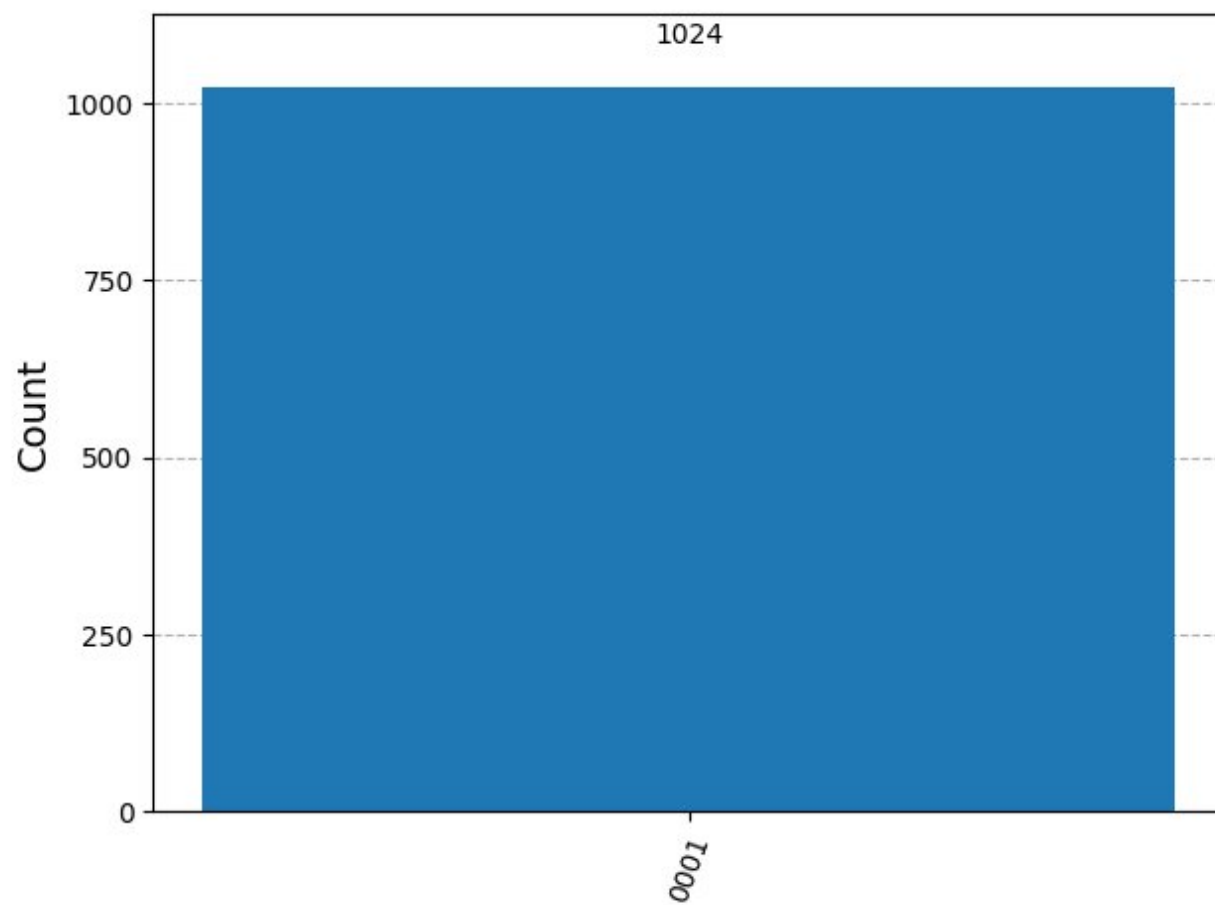
# Run on simulator
simulator = Aer.get_backend('qasm_simulator')
compiled_circuit = transpile(qc, simulator)
result = simulator.run(compiled_circuit, shots=1024).result()
counts = result.get_counts()
plot_histogram(counts)
plt.show()

found_a = max(counts, key=counts.get)
print("Algorithm found a =", found_a)

```

Secret string a: 0001





Algorithm found a = 0001